

VRP

Решение

Отжиг

Состояние

Поясню логику выбора. Как обычно в отжиге, мы хотим, чтобы невалидные состояния были непредставимы в нашем пространстве. Также мы хотим чтобы пространство допускало достаточно равномерную параметризацию, но при этом не слишком сильно размножало число состояний.

Задача чем-то похожа на set cover, поэтому первая идея – хранить два маппинга:

- маппинг машин на клиентов
- порядок объезда машин клиентами

Но хранить два объекта крайне некрасиво с точки зрения равномерной параметризации, и их ещё и нужно держать постоянно согласованными. Как мы знаем, в перестановки можно вложить всё что угодно, поэтому давайте сделаем одну большую перестановку клиентов. Ясно, что конкретные машины могут теперь обозначаться отрезками в этой перестановке клиентов. Уже неплохо, но это всё ещё два объекта которые нужно согласовывать и для которых нужно придумывать равномерную параметризацию переходов.

Давайте теперь избавимся и от этого, применив классический школьный трюк с шарами и перегородками и скажем, что машины тоже лежат в той же перестановке и съедают в себя всех людишек с предыдущей машины до себя.

Это заодно позволит нам немножко сэкономить симметрии (машины-то одинаковые!).

Итоговое состояние: перестановка из $n - 1$ клиента и V одинаковых маркеров машин. Маркер машины – первая точка из входа, то есть склад.

Шедулинг

Экспоненциальный, множитель 0.9999998, замерзаем в 0.0001. Вероятность принятия обычная.

Переходы

Равновероятно:

- Свапнуть два элемента перестановки
- Поревёрсить отрезок перестановки
- Удалить отрезок внутри одной машины и жадно его перестроить, по очереди вставляя точки в наиболее оптимальный для них участок пути.

Cost-функция

Отнормированная метрика расстояния, чтобы средняя длина ребра в MST равнялась 1

Старт

Поскольку у нас есть довольно неприятное ограничение на ровно V машин, тут простая жадина не подойдёт. Но датасеты достаточно малы, поэтому можно для старта понабирать рюкзаков дпшкой: сначала набрать самый жирный возможный, убрать элементы из рассмотрения, и повторить жадно.

Можно было бы и честное DP запустить, но и так работает на всех датасетах.

Шедулинг

Экспоненциальный, множитель 0.9999998, замерзаем в 0.0001. Вероятность принятия обычная.

Первая итерация

Отнормируем скор чтобы средний вес ребра в MST был равен 1. Добавим переходы: либо свапнуть позиции, либо реверснуть отрезок, вероятность каждого $\frac{1}{2}$.

Добавим нарушение капасити машин в кост с большим весом, чтобы отжиг не застревал в локальной неоптимальности стартового состояния.

Instance	Score	CPU time, s	Points
vrp_16_3_1	455.78	7.23	0
vrp_26_8_1	1 104.04	10.77	0
vrp_51_5_1	801.24	12.00	0
vrp_101_10_1	1 558.43	23.99	0
vrp_200_16_1	2 292.58	36.28	3
vrp_421_41_1	1 896.40	68.93	5

Итого: 8 / 30.

Вторая итерация

Научимся ещё и удалять случайный отрезок со средней длиной $\log_2(n)$ внутри маршрута одной машины (как в TSP), потом жадно его вставлять. Равновероятно с предыдущими переходами.

Instance	Score	CPU time, s	Points
vrp_16_3_1	455.78	11.33	0
vrp_26_8_1	1 104.04	15.44	0
vrp_51_5_1	800.52	20.13	0
vrp_101_10_1	1 560.56	36.58	0
vrp_200_16_1	2 339.24	52.81	3
vrp_421_41_1	1 909.95	95.92	5

Итого: 8 / 30. Не помогло и стало медленнее.

Третья итерация

Оказалось, что первая точка во входе – это склад, а не клиент с нулевым весом. Исправил парсер, всё полетело. Запретил нарушать капасити в промежуточных состояниях, так как это не влияет на итоговый скор.

Сиды: 958 420 773, 521 357 970, 4 003 061 857.

Instance	Score 1	Score 2	Score 3	Mean CPU, s	Median points
vrp_16_3_1	278.73	278.73	278.73	20.32	5
vrp_26_8_1	607.65	607.65	607.65	32.25	5
vrp_51_5_1	524.61	524.61	524.61	38.78	5
vrp_101_10_1	819.56	819.56	819.56	57.47	5
vrp_200_16_1	1 411.02	1 364.55	1 376.51	139.65	5
vrp_421_41_1	1 888.30	1 903.95	1 888.75	193.91	5

Итого: 30 / 30.